A Compact Row Storage Scheme for Cholesky Factors Using Elimination Trees

JOSEPH W. H. LIU
York University
Ontario, Canada

For a given sparse symmetric positive definite matrix, a compact row-oriented storage scheme for its Cholesky factor is introduced. The scheme is based on the structure of an elimination tree defined for the given matrix. This new storage scheme has the distinct advantage of having the amount of overhead storage required for indexing always bounded by the number of nonzeros in the original matrix. The structural representation may be viewed as storing the minimal structure of the given matrix that will preserve the symbolic Cholesky factor. Experimental results on practical problems indicate that the amount of savings in overhead storage can be substantial when compared with Sherman's compressed column storage scheme.

Categories and Subject Descriptors: G.1.3 [**Numerical Analysis**]: Numerical Linear Algebra—*sparse and very large systems*; G.4 [**Mathematics of Computing**]: Mathematical Software—*algorithm analysis*

General Terms: Algorithms

Additional Key Words and Phrases: Cholesky factorization, elimination tree, row storage, sparse matrix

1. INTRODUCTION

In this paper, we consider the direct solution of a large sparse symmetric positive definite linear system:

$$A\,x = b.$$

Robust algorithms and software have been developed to solve such systems. They typically involve the Cholesky factorization of the given sparse matrix A into LL^T, where L, the Cholesky factor, is a lower triangular matrix. To avoid the storage of and operations on zeros in the factor matrix L, some highly efficient

Research was supported in part by the Canadian Natural Sciences and Engineering Research Council under grant A5509, by the Applied Mathematical Sciences Research Program, Office of Energy Research, U.S. Department of Energy, under contract DE-AC05-84OR21400 with Martin Marietta Energy Systems Inc., and by the U.S. Air Force Office of Scientific Research under contract AFOSR-ISSA-84-00056.

Author's address: Department of Computer Science, York University, Downsview, Ontario, Canada M3J 1P3.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.
© 1986 ACM 0098-3500/86/0600-0127 $00.75

data structures are employed to store L [8, 12, 20, 21, 22]. They are mainly variants of the compressed column storage scheme proposed by Sherman [22], in which nonzeros in L are stored column by column together with an auxiliary vector keeping the index information (row subscripts) in a compressed form.

In this paper, we propose a new compact storage scheme for the Cholesky factor, in which nonzeros in L are stored row by row. Additional indexing information is also required to keep the column subscripts for the nonzeros in these rows. The data structure involved is based on the notion of *elimination trees* defined for sparse matrices. The structure may be regarded as a generalization of the envelope or profile storage method, so that zeros within the envelope are also exploited.

An outline of this paper follows. In Section 2, we define an elimination tree formally in terms of the elimination (node) sequence of a given sparse matrix. A brief review of its use in sparse factorization is given. We also establish some properties of the elimination tree that are relevant to the compact implicit row storage scheme.

The proposed storage scheme for the Cholesky factor is described in Section 3. We prove the important property that the amount of overhead storage required for indexing in this implicit scheme is always bounded above by the number of nonzeros in the *original* matrix structure. This property is not shared by any one of the existing storage methods, including Sherman's compressed column storage scheme [22]. An efficient algorithm is also presented to generate the row structure (i.e., the column subscripts of the nonzeros in the row), and the algorithm is not substantially more expensive than retrieving the subscripts from an array.

Also included in Section 3 is an interesting interpretation of this structural representation. It can be viewed as storing the minimal structure of the given matrix A, that will preserve the symbolic Cholesky factor structure. In graph-theoretic terms, edges in the symmetric graph of A are discarded provided that the structure of the filled graph is not affected. The reduced graph, which is represented in the storage scheme, is called the *skeleton graph* of A in this paper.

To use the storage scheme effectively, it is important to have efficient algorithms to generate the required data structures. Section 4 contains all the necessary algorithms to symbolically manipulate the sparse matrix problem in order to set up the required index vectors for the storage scheme.

In Section 5, the proposed storage scheme is analyzed for full envelope systems and the k-by-k grid model problem. Section 6 contains experimental runs on some practical problems. The results are compared with Sherman's compressed column storage scheme. Substantial savings in overhead storage for indexing are reported. Indeed, the compressed column scheme often requires from *two* to *four* times more subscript storage than the proposed row scheme. Section 7 contains our concluding remarks.

2. ON ELIMINATION TREES

2.1 Definition and Computer Representation of an Elimination Tree

In this paper, readers are assumed to be familiar with the graph-theoretic terminology used in the study of sparse elimination: fill, ordering, elimination graph model, reachable sets, and other related concepts. All the necessary

information can be found in [12]. Moreover, basic terminology and concepts in the study of trees are assumed; readers are referred to [1].

Consider the Cholesky factorization of a given sparse symmetric n-by-n positive definite matrix A into LL^T. Assume that the matrix A has already been ordered by some fill-reducing symmetric permutation (e.g., the minimum degree ordering [19] or the nested dissection ordering [9]). Let $G(A) = (X(A), E(A))$ be the symmetric graph associated with the sparse matrix A, and let $x_1, x_2, \ldots, x_n$ be the corresponding node ordering. Also, let $G(F) = (X(A), E(F))$, where $F = L + L^T$, be the associated *filled graph* for A.

The modelling of Gaussian elimination (for symmetric factorization) by graph structures is well known in the literature; the foundation of this approach was provided by Parter [17] and Rose [19]. In this paper, we use the notion of reachable sets that can be used to characterize $G(F)$ in terms of $G(A)$.

THEOREM 2.1. [12] *Let $j < i$.*

$$l_{ij} \neq 0 \text{ if and only if } x_i \in Reach_{G(A)}(x_j, \{x_1, \ldots, x_{j-1}\}).$$

A complete characterization of the structure of the resulting filled matrix F of A can be obtained in terms of the *Reach* operator[1] on the original matrix structure A. When matrix A is clear from the context, we use *Reach* instead of $Reach_{G(A)}$ for this operator.

We formally define an elimination tree, using our notation, as follows. For simplicity, we assume that the graph $G(A)$ is connected, or equivalently, that matrix A is irreducible. The results in this paper can be extended to disconnected graphs. The *elimination tree* for A is the directed tree

$$T(A) = (X(A), E_t(A)),$$

where x_n is the root of the tree, and $(x_j, x_i) \in E_t(A)$ if and only if $i = \min\{r \mid x_r \in Reach(x_j, \{x_1, \ldots, x_{j-1}\})\}$.

In matrix terms, $i = \min\{r \mid l_{rj} \neq 0\}$. That $T(A)$ is a directed tree follows from the fact that it is connected and has $n - 1$ edges in $E_t(A)$. The example in Figure 1 illustrates an elimination tree for a given 6-by-6 symmetric matrix. The structure of the tree $T(A)$ can be obtained readily by determining all the reachable sets:

$$\begin{aligned}
Reach(x_1, S_0) &= \{x_3, x_5\} \\
Reach(x_2, S_1) &= \{x_5, x_6\} \\
Reach(x_3, S_2) &= \{x_4, x_5\} \\
Reach(x_4, S_3) &= \{x_5, x_6\} \\
Reach(x_5, S_4) &= \{x_6\} \\
Reach(x_6, S_5) &= \varnothing.
\end{aligned}$$

Here and in the sequel, we shall use S_j to denote the set $\{x_1, \ldots, x_j\}$. We shall show in Section 4.2 that the elimination tree structure can be generated very efficiently without having to determine the reachable sets.

[1] For $x \in X(A)$ and $S \subseteq X(A)$, the reachable set is given by
$Reach(x, S) = \{y \in X(A) - S \mid$ there is a path from x to y through nodes in the subset $S\}$.

$$A = \begin{pmatrix} d & & x & & x & \\ & d & & & x & x \\ x & & d & x & x & \\ & & x & d & & x \\ x & x & x & & d & \\ & x & & x & & d \end{pmatrix}$$

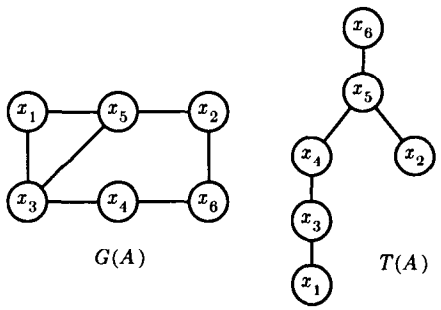


Fig. 1. An elimination tree for a 6×6 matrix.

Elimination trees have been used either implicitly or explicitly in the literature in the study of sparse symmetric factorization. The term *elimination tree* was first used by Duff [2]. The *element merge tree* used by Eisenstat, Schultz, and Sherman [6] in the element model has a structure close to that of an elimination tree. They use the tree again in [7] to devise the minimal storage sparse elimination algorithm. Duff and Reid [5] use an *assembly tree* to determine the assembly order for nodes in their multifrontal scheme, and its structure is a variant of an elimination tree structure that is improved for their application.

The efficient symbolic factorization scheme devised by George and Liu [11] makes use of the structure of an elimination tree. Schreiber [21] explicitly uses the notion of elimination trees in setting up his relative row-index scheme for storing the index overhead information for Cholesky factors. Jess and Kees [13] study the parallel decomposition of a sparse matrix using the model of an elimination tree. More recently, Duff [3] uses the tree for parallel multifrontal schemes, and Liu [15] uses it to study the task scheduling problem in parallel sparse Cholesky factorization on a shared-memory multiprocessor. Peters calls the tree a *palm* and uses it to study preserving partitions. The *row merge tree* used by Liu [14] for sparse orthogonal factorization is also closely related to this notion.

The elimination tree as defined provides a valuable tool in the study of sparse elimination. Its structure is simple and economical to store. Indeed, any rooted tree (and hence an elimination tree) can be represented conveniently using the $PARENT[*]$ vector. For each node x_i with $i < n$,

$$x_{PARENT[i]}$$

is defined to be the parent node of x_i in the elimination tree. For the root x_n, we let

$$PARENT[n] = 0.$$

The *PARENT* vector for the elimination tree of Figure 1 is given as follows:

i	1	2	3	4	5	6
PARENT[i]	3	5	4	5	6	0

Therefore, the representation requires only a *PARENT* vector of size n. More important, the representation contains a lot of information pertinent to the elimination sequence and, hence, about the fill-in for the given sparse matrix. The next section will explore some of its properties that are relevant to the compact row storage scheme.

2.2 Row Structure of the Cholesky Factor in Terms of its Elimination Tree

We assume that readers are familiar with the notions of *ancestors* and *descendants* in a tree. By the *subtree rooted at* a node x or simply the subtree at x, we mean the subgraph consisting of the node x together with all its descendants in the tree. A *pruned subtree* at the node x refers to the subgraph obtained after the removal of some subtrees from the subtree rooted at x.

Schreiber [21] provides a number of interesting properties of elimination trees that are relevant to the study of sparse elimination. In particular, he gives a partial characterization on the row structures of the Cholesky factor L. He observes that each row structure of L is associated with a pruned subtree in the elimination tree. To aid our discussion, we shall refer to the pruned subtree rooted at x_i, containing nodes defined by the structure of the ith row of L, as the *row subtree* at x_i. We quote the following theorem from Schreiber ([21], page 260, Proposition 5).

THEOREM 2.2. [21] *If $l_{ij} \neq 0$ $(j < i)$, then x_i is an ancestor of x_j in the elimination tree.*

The next theorem relates nonzeros in the Cholesky factor with the structure of the elimination tree. This result is central to the row storage scheme proposed in the next section. We first need a lemma, which provides a characterization of the ancestor–descendant relation in the elimination tree.

LEMMA 2.3. *Let $k < j$. The node x_j is an ancestor of x_k in the elimination tree if and only if they are connected by a path in the subgraph of $G(A)$ containing the nodes $S_j = \{x_1, \ldots, x_j\}$.*

PROOF. *If Part:* Assume that there is a path between x_j and x_k in the subgraph. We prove the result by induction on the length of the path. If the length is 1, that is $a_{jk} \neq 0$, this implies that $l_{jk} \neq 0$, so that by Theorem 2.2, x_j is an ancestor of x_k. Assume that the result holds for path lengths less than t, and the path under consideration is of length t. Let x_m be the node on this path with the largest subscript value. If $m < k$ then $l_{jk} \neq 0$ (by Theorem 2.1), so that x_j is an ancestor of x_k (by Theorem 2.2). If $m \geq k$, by the induction assumption, x_j is an ancestor of x_m, which is in turn an ancestor of x_k. Therefore x_j must be an ancestor of x_k.

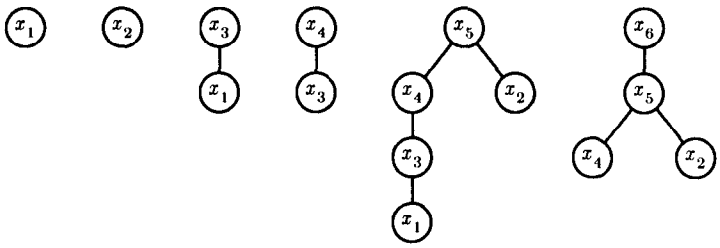


Fig. 2. Row subtrees for the example in Figure 1.

Only If Part: Assume that x_j is an ancestor of x_k. Consider the path x_k, $x_{PARENT[k]}$, $x_{[PARENT]PARENT[k]]}$, ..., x_j in the elimination tree. Each consecutive pair corresponds to a nonzero in the factor matrix L, so that by Theorem 2.1, there exists a path joining each pair of nodes going through nodes in subsets of S_{j-1}. Joining these paths together, we obtain a path from x_j to x_k in the subgraph containing nodes in S_j. □

THEOREM 2.4. Let $j < i$. $l_{ij} \neq 0$ *if and only if* x_j *is an ancestor of some node* x_k, *where* $a_{ik} \neq 0$.

PROOF. *If Part*: Assume that x_j is an ancestor of some x_k, with $a_{ik} \neq 0$. Obviously, $k \leq j$. If $k = j$, there is nothing to prove. If $k < j$, by Lemma 2.3, there is a path from x_j to x_k through nodes in S_{j-1}. Together with the edge $\{x_i, x_k\}$, we obtain a path from x_j to x_i through nodes in S_{j-1}. Using Theorem 2.1, we have $l_{ij} \neq 0$.

Only If Part: Assume that $l_{ij} \neq 0$. By Theorem 2.1, there exists a path x_j, x_{j_1}, ..., x_{j_t}, x_i in the graph $G(A)$, where $j_s < j$, for $s = 1, \ldots, t$. If $t = 0$, the result follows since we have $a_{ij} \neq 0$, and x_j is an ancestor of itself in the tree. If $t > 0$, let $k = j_t$. Obviously, $a_{ik} \neq 0$, and by Lemma 2.3, x_j is an ancestor of x_k in the tree. □

Theorem 2.4 implies that the row subtree at x_i includes all nodes on paths in the elimination tree from x_j to x_i with $a_{ij} \neq 0$. Each row subtree will be completely determined by the set of its leaves. The next corollary characterizes such leaf nodes. These nodes specify the locations in which the subtree rooted at x_i should be pruned.

COROLLARY 2.5. *The node* x_j *is a leaf in the row subtree at* x_i *if and only if* $a_{ij} \neq 0$, *and, for every descendant* x_k *of* x_j, $a_{ik} = 0$.

We shall illustrate the result using the matrix example of Figure 1. We give the row subtree of each row of L in Figure 2.

3. COMPACT ROW STORAGE SCHEME

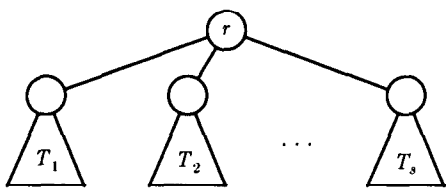
3.1 Storage Scheme Using Elimination Tree

The result of Corollary 2.5 provides the basis for a compact row-oriented storage scheme for the Cholesky factor of a given sparse matrix. It is shown that the row structure of L_{i*} is uniquely characterized by the pruned subtree at x_i defined by

its leaf nodes in Corollary 2.5. We can hence represent the structure of L in an implicit form by storing the elimination tree and the set of leaf nodes for each row subtree in the $PARENT[*]$ vector.

This implicit structural representation can be used to generate the subscript sequence for each row of L when required. In practice, it is desirable to be able to generate the sequence in *ascending* order of the subscripts, since that will be the sequence in which nonzeros in each row of the Cholesky factor are stored. To facilitate the generation of this subscript sequence, we consider an equivalent reordering of the matrix.

It is known in the literature [5, 7, 18] that given an ordering of a sparse matrix, any reordering that numbers a node ahead of its parent node in its elimination tree is equivalent to the original ordering in terms of fills and computation. In particular, a *postordering* [1] will give an equivalent reordering of this type. The postorder sequence of a rooted tree T:



can be generated by the following recursive algorithm.

Algorithm 3.1. Postordering(T)
if $s = 0$
then sequence is r
else sequence is Postordering(T_1), Postordering(T_2), . . . , Postordering(T_s), r.

It will become clear that a postordering is appropriate for the storage scheme when the row structure generation algorithm is discussed in Section 3.2. In what follows, we assume that the ordering $\{x_i\}$ is already a postordering of the elimination tree. Figure 3 contains the same matrix example as Figure 2, except that the ordering $\{x_i\}$ is now a postordering of its elimination tree.

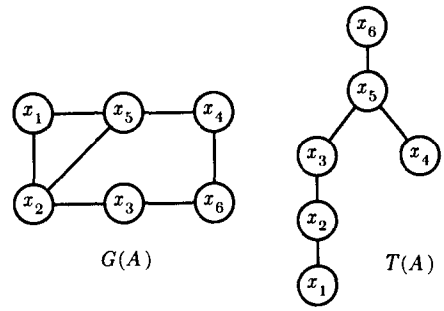
Figure 4 illustrates the corresponding storage vectors for this matrix example. The vector $LEAF[*]$ stores the list of leaves for each row subtree, and the index vector $XLEAF[*]$ identifies the beginning of each list. Therefore, the leaves that define the subtree of row i are given by $LEAF[j]$, for $j = XLEAF[i], \ldots, XLEAF[i+1] - 1$. The vector $XLEAF[*]$ is of size $n + 1$, and the size of $LEAF[*]$ depends on the matrix problem.

In the storage scheme, the diagonal elements of A are stored in the real vector $DIAG[*]$. Entries in the lower triangle of A are stored row by row in the real vector $LNZR[*]$, with zeros inserted into the data structure to accommodate for fills in the Cholesky factor. Note that in Figure 4, two zeros are stored in $LNZR$ for the fills at (5, 3) and (6, 5) in the factor matrix. An additional index vector $XLNZR[*]$ is required to mark the beginning of nonzeros for each row. Altogether, the overhead storage required for indexing information is $3n + 2$, plus the storage taken by the vector $LEAF[*]$.

THEOREM 3.1. *Size of the vector* $LEAF \leq |E(A)|$.

$$A = \begin{pmatrix} d & x & & & x & \\ x & d & x & & x & \\ & x & d & & & x \\ & & & d & x & x \\ x & x & & x & d & \\ & & x & x & & d \end{pmatrix}$$

Fig. 3. A postordering of an elimination tree.



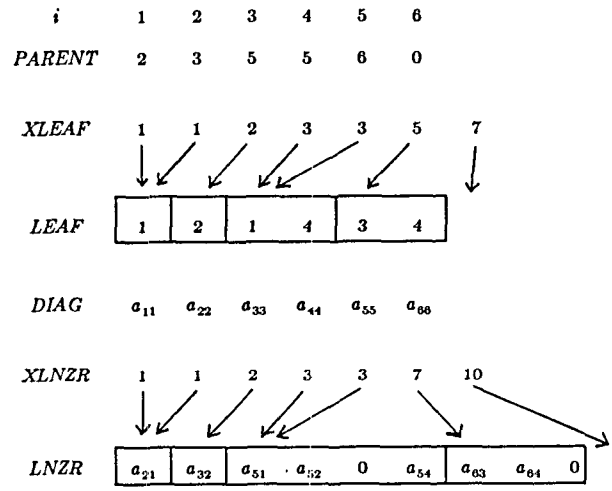


Fig. 4. Compact row storage scheme for Figure 3.

PROOF. For each leaf node x_j of the row subtree rooted at x_i, we have $j < i$ and $\{x_j, x_i\} \in E(A)$. The result then follows. □

Theorem 3.1 provides an important property of this storage scheme: the overhead storage for indexing is bounded by $3n + 2$, plus the number of nonzeros in the original matrix A. It should be noted that this property is not shared by any of the existing storage schemes [21, 22]. Moreover, in practice, the size of $LEAF[*]$ is much smaller than $|E(A)|$. Examples will be presented later to illustrate the possible difference.

3.2 Row Structure Generation Algorithm

As noted before, for the row storage scheme in Section 3.1 to be effective, we need an efficient algorithm to generate the row structures. Specifically, for each row i, we want to efficiently generate the subscript sequence in the same order as the nonzeros of L_{i*} are stored; that is, in increasing order of subscripts. The given matrix is assumed to be ordered by a postordering sequence of its elimination tree.

The following algorithm performs a postorder traversal [1] of the row subtree at x_i, and it generates the correct sequence of column subscripts for nonzeros in the ith row L_{i*}. At the end of the algorithm, *Row-structure* contains the required sequence of subscripts.

Algorithm 3.2. Row i Structure Generation

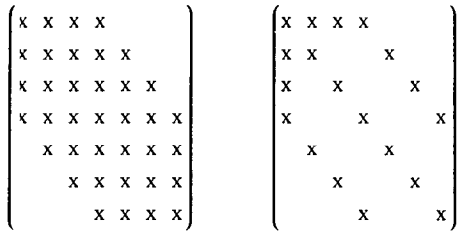
```
Row-structure := ∅;
for j := XLEAF[i] to XLEAF[i + 1] − 1 do
begin
  node := LEAF[j];
  if j + 1 < XLEAF[i + 1]
    then nextleaf := LEAF[j + 1]
    else nextleaf := i;
  while node < nextleaf do
  begin
    Row-structure := Row-structure ∪ {node};
    node := PARENT[node]
  end
end;
```

General implementations of postorder traversal on a tree usually require either recursion or the manipulation of an explicit stack. Algorithm 3.2 generates the correct sequence of column subscripts in increasing order because the rows and columns of A are assumed to be ordered by a postordering sequence of the elimination tree. This sequence explains why no stack or recursion is necessary in the algorithm to backtrack the traversal.

Since the entries in each row of L are often processed sequentially, and they are stored in increasing order of column subscripts, Algorithm 3.2 can be used to provide the structural information in conjunction with row operations.

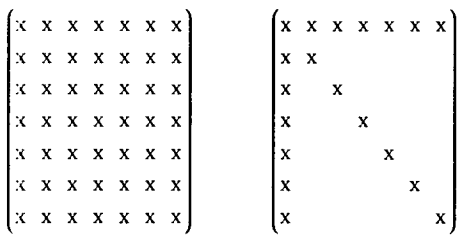
3.3 Matrix and Graph-Theoretic Interpretation on Storage Scheme

We now provide some interesting interpretations of the structure (*XLEAF*, *LEAF*) as described in the row storage scheme. The scheme can be viewed as a generalization of the *profile* or *envelope* method [12] for sparse symmetric matrices. In the envelope method, the overhead storage requires one integer per row to keep the column subscript of the first nonzero, and zeros within the envelope are not exploited. The proposed row storage scheme generalizes the envelope storage method so that every zero in a row is exploited. This is achieved by maintaining the locations of a subset of nonzeros in the original matrix for each row as given by (*XLEAF*, *LEAF*). They provide entry points from which other nonzeros in the Cholesky factor can be generated.



Banded matrix and its skeleton matrix

Fig. 5. Examples of skeleton matrices.



Full matrix and its skeleton matrix

In what follows, we shall provide an interpretation of the row scheme in terms of the skeleton matrix and skeleton graph to be defined. Let A be the given sparse symmetric matrix, and F be its filled matrix. Define the symmetric matrix A^- as follows:

For each row i, $\quad a^-_{ii} = a_{ii}$,

$$\text{And for } j < i, \qquad a^-_{ij} = \begin{cases} a_{ij} & \text{if } x_j \text{ is a leaf of row subtree at } x_i \\ 0 & \text{otherwise} \end{cases}$$

In other words, we discard those off-diagonal nonzeros a_{ij} in the matrix A if the node x_j has a descendant x_k with $a_{ik} \neq 0$. Such nonzero location j is represented implicitly. Let

$$G(A^-) = (X(A), E(A^-))$$

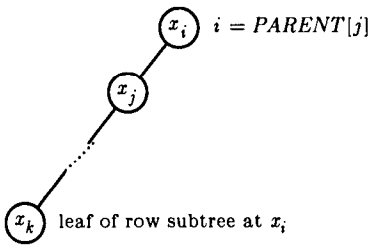
be the corresponding symmetric graph of $G(A^-)$.

We shall call A^- the *skeleton matrix* and $G(A^-)$ the *skeleton graph* of A, respectively. It should be clear that the storage vectors $(XLEAF, LEAF)$ give the exact adjacency structure of (the lower triangle of) A^-. Figure 5 illustrates the structures of the skeleton matrices for a banded matrix and a full matrix. Some interesting properties of the skeleton graph are established in the rest of this subsection.

LEMMA 3.2 $\quad x_{PARENT[j]} \in Reach_{G(A^-)}(x_j, S_{j-1})$.

PROOF. Put $i = PARENT[j]$. We shall prove by induction on j. The result is obviously true for $j = 1$, since $\{x_i, x_j\} \in E(A^-)$. Assume that the result holds for all values less than j.

We now prove the result for the value j. If $\{x_i, x_j\} \in E(A^-)$, there is nothing to prove. Assume that this is not true. Since the edge $\{x_i, x_j\}$ belongs to the row subtree at x_i, there must exist a leaf node x_k of this row subtree, with $a_{ik} \neq 0$. Obviously, $k < j < i$. And, by the definition of $G(A^-)$, $\{x_k, x_i\} \in E(A^-)$.



By the inductive assumption, we have

$$x_{PARENT[k]} \in Reach_{G(A^-)}(x_k, S_{k-1}),$$

$$x_{PARENT[PARENT[k]]} \in Reach_{G(A^-)}(x_{PARENT[k]}, S_{PARENT[k]-1})$$

$$\cdots$$

and so on, until the node x_j. Combining these paths with the edge $\{x_k, x_i\}$ in $E(A^-)$, we can obtain a path from x_i to x_j going through nodes in the subset $\{x_1, \ldots, x_{j-1}\}$ in the skeleton graph $G(A^-)$, and hence the result follows. □

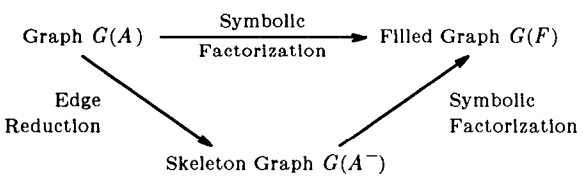
THEOREM 3.3 $Reach_{G(A)}(x_j, S_{j-1}) = Reach_{G(A^-)}(x_j, S_{j-1})$.

PROOF. The result can be proved by induction on j with the use of the result in Lemma 3.2 and arguments similar to those for the proof of Lemma 3.2. The proof will be left for the reader. □

COROLLARY 3.4 $T(A) = T(A^-)$.

COROLLARY 3.5. *If $G(F^-)$ is the (structural) filled graph of $G(A^-)$, then $G(F) = G(F^-)$.*

The result in Corollary 3.5 has the following interesting interpretation. The filled graph $G(F)$ is uniquely determined by the structure of the symmetric graph $G(A)$, and the filled graph can be obtained from a process called symbolic factorization. However, some edges in $G(A)$ can be removed without affecting the structure of this filled graph. Indeed, the graph structure $G(A^-)$ represents the minimum that we have to retain in order to preserve the structure of the filled graph. Their relations are presented diagramatically as follows:



It is interesting to note that the connection between the filled graph and the skeleton graph of a matrix graph is analogous to that between the transitive closure and the transitive reduction of directed graphs [1].

4. Data Structure Setup

4.1 Overview

Section 3 contains a compact row storage scheme for Cholesky factors. To implement this scheme, the data structure containing the *PARENT* vector of the elimination tree, the compact representation (*XLEAF*, *LEAF*) of the skeleton graph, and the row indexing vector *XLNZR* must all be determined. Given the initial ordering of the matrix and the matrix structure, the overall data structure setup consists of the following major steps:

Step 1. Determination of the elimination tree in the form of the vector *PARENT*[*] from the initial ordering and matrix structure.

Step 2. Determination of a postordering for the elimination tree.

Step 3. Determination of the skeleton graph structure for the matrix reordered by the postordering sequence.

Step 4. Determination of the index vector for rows of the Cholesky factor L of the reordered matrix.

Each of these four steps will be examined in the following subsections.

4.2 Determination of the Elimination Tree

Given the adjacency structure of a sparse matrix and an elimination sequence, the following algorithm will generate the structure of the corresponding elimination tree in the form of the *PARENT*[*] vector.

Algorithm 4.1. Elimination Tree

```
for i := 1 to n do
begin
  PARENT[i] := 0;
  for x_k ∈ Adj(x_i) and k < i do
  begin
    /* Find the root x_r of the tree in the forest containing the node x_k */ r := k;
    while (PARENT[r] ≠ 0) and (PARENT[r] ≠ i) do
      r := PARENT[r];
    if (PARENT[r] = 0) then PARENT[r] := i;
  end
end
```

Here, $Adj(x)$ is used to represent the set of nodes adjacent to the node x in the graph $G(A)$. At step i of this algorithm, the partial elimination tree (strictly speaking, forest) for the first i nodes $x_1, \ldots, x_i$ will be formed. The possibly time-consuming part is in the while loop of Algorithm 4.1, where paths are traced bottom-up to identify all the children of the current node x_i in the elimination tree.

The algorithm can be speeded up by introducing some form of *path compression.* This can be accomplished by using a temporary vector $ANCESTOR[*]$. The modified algorithm can be stated as follows:

Algorithm 4.2. Elimination Tree by Path Compression

```
for i := 1 to n do
begin
  PARENT[i] := 0;
  ANCESTOR[i] := 0;
  for x_k ∈ Adj(x_i) and k < i do
  begin
    r := k;
    while (ANCESTOR[r] ≠ 0) and (ANCESTOR[r] ≠ i) do
    begin
      t := ANCESTOR[r];
      ANCESTOR[r] := i;
      r := t
    end;
    if (ANCESTOR[r] = 0) then
    begin
      ANCESTOR[r] := i;
      PARENT[r] := i
    end
  end
end
```

The algorithm maintains two structures side by side as defined by the current values of *PARENT* and *ANCESTOR.* The *ANCESTOR* vector provides shortcuts in determining the root of the present subtree containing the node x_k. In the literature, the tree (forest), defined by the *ANCESTOR* vector in Algorithm 4.2, has been referred to as the *virtual tree* (*virtual forest*). Similar techniques have been used by Duff and Reid [5] to generate the tree. Figure 6 shows the various stages in executing Algorithm 4.2 on the example in Figure 1.

In [24], Tarjan gives an excellent survey of tree manipulation algorithms for the set union problem. Our problem of finding the *PARENT* vector from $G(A)$ can be expressed quite nicely in terms of the basic set operations. On applying the complexity analysis result from [23], it can be established that $O(|E(A)| \log_2 n)$ is an upper bound on the complexity of Algorithm 4.2. There is yet a more efficient algorithm, which Tarjan calls *path compression with balancing.* Interested readers are referred to [24] for detail.

4.3 Determination of a Postordering

Given an initial ordering of a sparse symmetric matrix, the algorithms in the previous section determine the structure of the corresponding elimination tree. A desirable equivalent ordering (equivalent with respect to the number of fills and arithmetic operations) required for the compact implicit row storage scheme is the postordering of this elimination tree. It can be generated by a recursive postorder traversal of the tree, and readers are referred to [1]. In [5], Duff and Reid also provide details for this task that is formulated as a depth-first search on the elimination tree. It should be clear that only the *PARENT* vector is necessary to produce a postordering.

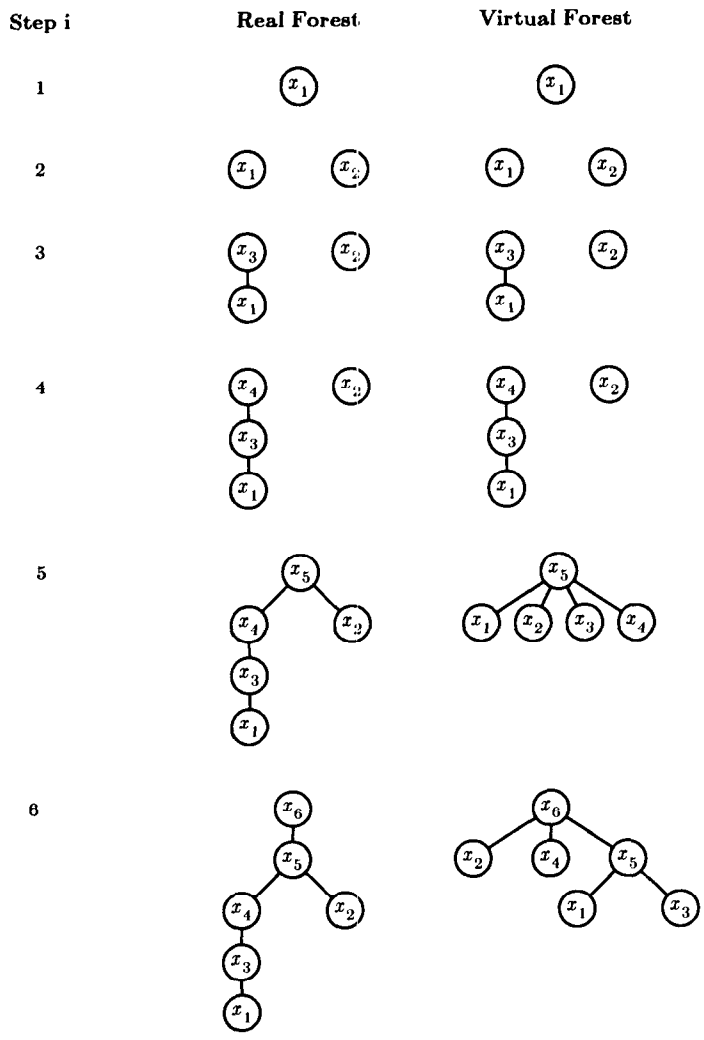


Fig. 6. Stages in the execution of Algorithm 4.2.

4.4 Determination of the Skeleton Graph

In this section, we consider the construction of the skeleton graph structure, given a postordering of the elimination tree. Let $x_1, x_2, \ldots, x_n$ be the given node postordering sequence, and let *PARENT* be the parent vector of its elimination tree.

Our objective here is to construct the skeleton graph that is completely specified by the leaves of all the row subtrees as given in the (*XLEAF*, *LEAF*) structure. Consider the row subtree rooted at x_i. Therorem 2.4 provides a characterization of this subtree and we use it to prove the next theorem.

THEOREM 4.1. *Let* $Adj(x_i) \cap \{x_1, \ldots, x_{i-1}\} = \{x_{c_1}, \ldots, x_{c_s}\}$, *with*

$$c_1 < c_2 < \cdots < c_s < i.$$

Then for $1 < j < s$, either x_{c_j} is a leaf node in the row subtree at x_i or x_{c_j} is an ancestor of $x_{c_{j-1}}$ in this subtree.

PROOF. If the node x_{c_j} is a leaf node in the row subtree at x_i, there is nothing to prove. Assume that it is not the case. Then, by Theorem 2.4, there is a descendant x_k of x_{c_j}, where $k < c_j$ and $a_{ik} \neq 0$. So, we have $k \leq c_{j-1} < c_j$ and x_k is a descendant node of x_{c_j}. Since the ordering is already a postordering of the elimination tree, every node x_s, with $k \leq s < c_j$ must be a descendant of x_{c_j}. It then follows that x_{c_j} is an ancestor of $x_{c_{j-1}}$. □

The result of Theorem 4.1 provides a scheme to determine the skeleton graph structure. For each node x_i, we can first order the neighbors of x_i with subscripts smaller than i, and then discard those that are ancestors of the previous one in the neighbor sequence. To determine the ancestor–descendant relation, we use the following result from [1].

THEOREM 4.2 *For any given rooted tree, a node x is a proper ancestor of y if and only if*

$$postorder[x] - number\ of\ proper\ descendants\ of\ x \leq postorder[y] < postorder[x],$$

where $postorder[x]$ is the location of x in the postordering sequence.

COROLLARY 4.3. *Let $Adj(x_i) \cap \{x_1, \ldots, x_{i-1}\} = \{x_{c_1}, \ldots, x_{c_s}\}$, with*

$$c_1 < c_2 < \cdots < c_s < i.$$

Then $\{x_{c_j}, x_i\} \in E(A^-)$ if and only if $j = 1$ or

$$postorder[x_{c_{j-1}}] < postorder[x_{c_j}] - number\ of\ proper\ descendants\ of\ x_{c_j}.$$

The use of Corollary 4.3 requires the number of proper descendants for each node in the elimination tree. Given the *PARENT* vector for the elimination tree and the postordering $x_1, \ldots, x_n$. It is simple and straightforward to devise an $O(n)$ algorithm to determine the required proper descendant counts.

4.5 Determination of the Row Index Vector

The data structure setup also requires the formation of the index vector *XLNZR* to mark the beginning of the rows of entries in the Cholesky factor L stored in the main storage vector *LNZR*. This index vector can be determined by simply counting the number of entries in each row of the factor matrix L. A minor modification of the row structure generation algorithm in Section 3.2 will do. It should be pointed out that the computational complexity of this step will be proportional to the number of entries in the factor matrix L.

5. STORAGE ANALYSIS ON SOME MODEL PROBLEMS

5.1 Band and Envelope Systems

In this section, we study the use of the implicit row storage scheme on different classes of problems. In particular, we consider systems with nonzero subdiagonals in the Cholesky factors. It is known that these systems have full envelopes [10], and full band systems obviously have this property.

Let A be such a matrix, that is, if L is its Cholesky factor, $l_{i,i-1}$ is nonzero for $i > 1$. It can be readily verified that the elimination tree $T(A)$ is simply a chain of nodes from x_n to x_{n-1}, and so on until x_1. Furthermore, the row subtree at x_i is itself a chain of nodes with x_{f_i} as the leaf node, where f_i is the subscript of the first nonzero in row i of the matrix A. Therefore the storage overhead to implement the implicit row storage scheme for such systems is $4n + 2$, as required by the vectors *PARENT*, *XLEAF*, *LEAF*, and *XLNZR*.

In the envelope or profile storage method for sparse symmetric matrices, the entries within the envelope are stored row by row. The structure of each row i is represented by the location f_i of the first nonzero. In this way, the row structure for row i of the factor matrix is given implicitly by $f_i, f_i + 1, \ldots, i - 1, i$. Note that in this method, zeros within the envelope are not exploited. It becomes clear that the proposed row scheme in this paper can be regarded as a generalization of the conventional envelope method.

5.2 The k-by-k Regular Grid

In this section, we study the use of the implicit row storage scheme for problems arising from the k-by-k regular grid. Consider the matrix A associated with the nine point difference operator on the k-by-k grid with the nested dissection ordering [9].

Figure 7 contains the nested dissection ordering for a 7-by-7 grid and the resulting elimination tree. The ordering shown is already a postordering defined on the elimination tree. In the figure, we have adopted the notation used by Schreiber [21], in which a double edge indicates a chain of nodes. In this case, the nodes on the chain have consecutive numbering.

For the separator defined by the nodes 43–49, we list their respective lists of leaf nodes as follows:

Node	Leaves			
43	4	22		
44	4	5	22	23
45	5	23		
46	5	13	23	31
47	13	31		
48	13	14	31	32
49	14	32		

Note also that the leaf nodes in the elimination tree, that is, 1, 2, 4, 5, 10, 11, . . . , all have trivial row subtrees.

THEOREM 5.1. $|E(A^-)| \leq 2k^2$.

PROOF. Consider the "+" separator for the k-by-k regular grid. Consecutive nodes along this separator will have subtrees with at most two and four leaves respectively. On the average, each node has a subtree with at most three leaves.

Note there are $k^2/4$ nodes in the last level of the elimination tree, whose row subtrees are trivial and do not contribute any edges to $E(A^-)$. Moreover, each separator starts with a node whose subtree has only two leaves. In total, we have

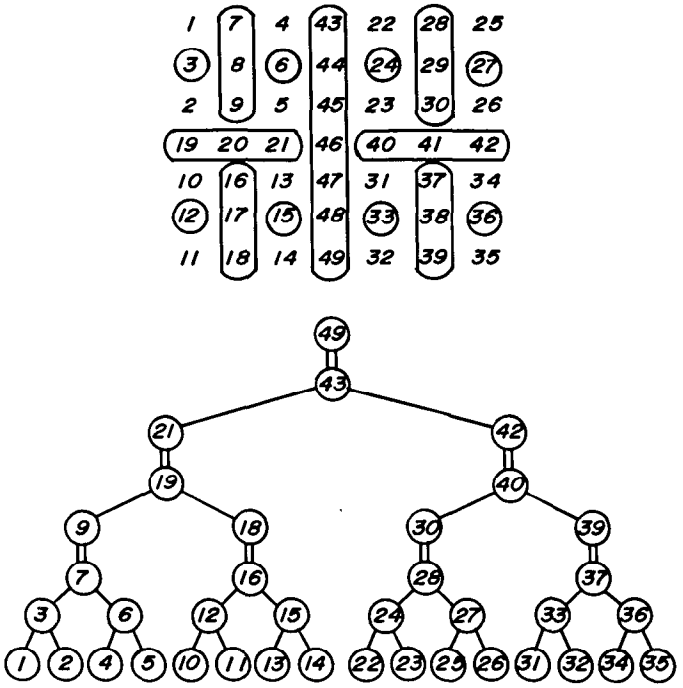


Fig. 7. Nested dissection ordering and its elimination tree on a 7×7 grid.

$k^2/4$ nodes with no leaves, $k^2/4$ nodes with two leaves, and the rest with an average of three leaves. Summing over the k^2 nodes, we have the required result. □

6. EXPERIMENTAL COMPARISON WITH SHERMAN'S COMPRESSED COLUMN STORAGE SCHEME

The most popular storage scheme for Cholesky factors of sparse matrices has been Sherman's compressed column storage scheme [22]. His scheme stores the entries in the Cholesky factor column by column. The subscript information is maintained in an auxiliary vector in a compressed form by taking advantage of columns whose row subscripts are final subsequences of those in the previous column.

This storage scheme is known to be very effective for band systems and for general sparse matrix problems. It has been shown that for the k-by-k grid problem, the associated compressed column subscripts require no more than $12k^2$ locations [11, 22]. In this section, we provide experimental results comparing the subscript storage requirement of Sherman's scheme with the compact row storage scheme proposed in this paper.

6.1 Nested Dissection on the k-by-k Model Problem

Theorem 5.1 provides an upper bound of $2k^2$ for the number of subscripts required by the compact row scheme for the k-by-k regular grid model problem with nested dissection ordering [9]. On the other hand, $12k^2$ has been established as an upper

Table I. Comparison of Subscript Storage for the k-by-k Model Problem with Nested Dissection

k	n	$\|E(A)\|$	$\|E(F)\|$	$\|E(A^-)\|$	Compressed column scheme	Compact row scheme
50	2500	9702	54636	4263	15074	6763
60	3600	14042	84912	6903	23134	10503
70	4900	19182	122981	9177	33639	14077
80	6400	25122	169667	11297	46147	17697
90	8100	31862	224467	13617	56742	21717

Table II. Selected Harwell-Boeing Test Matrices

Key	Number of equations	Description
BCSPWR09	1723	Symmetric structure of western U.S. power network
BCSPWR10	5300	Symmetric structure of entire U.S. power network
BCSSTK08	1074	Symmetric stiffness matrix, frame building (TV studio)
BCSSTK13	2003	Symmetric stiffness matrix, fluid flow general eigenvalues
BCSSTM13	2003	Symmetric mass matrix, fluid flow general eigenvalues
BLCKHOLE	2132	Connectivity structure of a geodesic dome on a coarse base
CAN 1072	1072	Symmetric pattern from Cannes, Lucien Marro
DWT 2680	2680	Symmetric connection table from DTNSRDC, Washington
LSHP3466	3466	Symmetric matrix from Alan George's L-SHAPE problems
FS 541 1	541	Unsymmetric facsimile convergence matrix
SHL 400	663	Unsymmetric basis from LP problem SHELL
BP 1600	822	Unsymmetric basis from LP problem BP

bound for Sherman's compressed column scheme [22]. In Table I, we tabulate results of experimental runs for various values of k. Since the row scheme requires an n-vector for $PARENT[*]$, its subscript storage requirement in the table, labeled as "Compact row scheme," is computed as $|E(A^-)| + n$. The advantage of the compact row scheme in terms of subscript storage is clear from the table. Indeed, for $k \geq 90$, the compressed column scheme requires about three times the amount of subscript storage than the row scheme.

6.2 Harwell-Boeing Sparse Matrix Test Problems

Problems from the Harwell-Boeing collection of sparse matrix examples [4] were also tested. They are used to demonstrate the performance of the compact row storage scheme on a variety of practical problems. Table II contains the list of problem examples selected, and we use the same keys as provided by the Harwell-Boeing tape to identify the different matrices. The problem subset has been carefully selected, and it is quite representative of the problems in the entire collection. The experimental results presented for these problems are also comparable to runs on other problems from the collection not displayed here.

Nine symmetric and three unsymmetric matrices are used. For the unsymmetric examples, we use the structure of $A + A^T$. For each matrix problem, the popular fill-reducing minimum degree algorithm is used to order the equations. The version used is the author's recent implementation of the modified ordering algorithm by multiple elimination [16].

Table III. Comparison of Subscript Storage for Selected Harwell-Boeing Examples

Problem	$\|E(A)\|$	$\|E(F)\|$	$\|E(A^-)\|$	Compressed column scheme	Column scheme (postorder)	Compact row scheme
BCSPWR09	2394	4592	2113	3877	3437	3936
BCSPWR10	8271	22764	7160	16796	15404	12460
BCSSTK08	5943	29973	3180	10842	10764	4254
BCSSTK13	40940	269668	5598	27962	27755	7601
BCSSTM13	9970	43491	1960	6915	6737	3963
BLCKHOLE	6370	51570	3256	13056	12975	5388
CAN 1072	5686	19329	2159	7091	7071	3231
DWT 2680	11173	53364	5364	18610	17839	8044
LSHP3466	10215	83116	5272	21208	20983	8738
FS 541 1	2463	6768	1258	4017	4017	1799
SHL 400	1709	14411	1355	5358	5314	2018
BP 1600	4809	63844	3141	13602	13585	3963

Table IV. Comparison of Data Structure Setup Time for Selected Harwell-Boeing Examples

Problem	Postorder time	Column scheme setup time	Skeleton graph	*XLNZR*	Row scheme setup time
BCSPWR09	0.32	0.46	0.25	0.12	0.37
BCSPWR10	1.12	1.79	0.88	0.41	1.29
BCSSTK08	0.42	0.89	2.08	0.15	2.23
BCSSTK13	2.02	6.49	6.91	0.34	7.25
BCSSTM13	0.66	1.24	1.41	0.14	1.55
BLCKHOLE	0.58	1.03	0.56	0.23	0.79
CAN 1072	0.40	0.70	0.62	0.13	0.75
DWT 2680	0.91	1.58	1.02	0.32	1.34
LSHP3466	0.95	1.73	0.91	0.38	1.29
FS 541 1	0.18	0.35	1.22	0.06	1.28
SHL 400	0.18	0.39	0.52	0.08	0.60
BP 1600	0.36	1.02	0.79	0.15	0.94

Table III compares the subscript storage requirement for the compact row and compressed column storage schemes. The column marked "Compressed column scheme" is the number of subscripts required by Sherman's scheme on the matrix problem ordered by the modified minimum degree ordering from [16]. The column labelled "Column scheme (postorder)" also contains the subscript requirement, except that the ordering is now a postordering of the minimum degree sequence. As in Table I, "Compact row scheme" is the total subscript storage requirement, computed as $|E(A^-)| + n$ to include the additional n-vector $PARENT[*]$.

It should be noted that there is some minor improvement in terms of reduced subscript storage if a postordering is used. On the other hand, the compact row storage scheme requires significantly less storage than the compressed column scheme (with or without the use of postordering). It ranges from two to five times less subscript storage. Although the row scheme requires an extra n-vector for $PARENT[*]$, the overall reduction is still substantial.

Table IV contains the running time to set up the data structures for the two storage schemes. The times reported are in seconds on a VAX 11/780. The column labelled "Column scheme setup time" contains the time required to

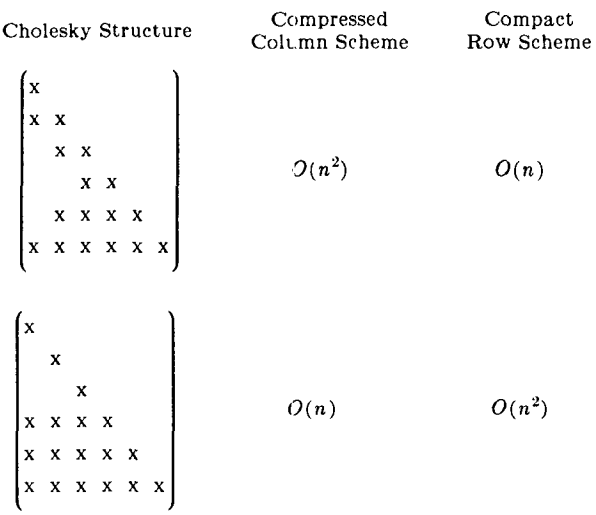


Fig. 8. Comparison of subscript storage for two contrived matrix structures.

perform the symbolic factorization to generate Sherman's compressed column scheme. The total time required to set up the compact row scheme ("Row scheme setup time") is the sum of the time to generate the skeleton graph structure and the time to generate the index row vector *XLNZR*. With the exception of the problems "BCSSTK08" and "FS 541 1," the times required to set up the two schemes are comparable.

7. CONCLUDING REMARKS

This paper provides the theoretical basis for a highly efficient storage scheme for the Cholesky factors of symmetric sparse matrices. The scheme is row-oriented and stores the row structures implicitly by making use of the elimination tree. The study also relates the storage scheme to the interesting structure of a skeleton graph. It is a subgraph of the original graph structure, and it represents the minimal subgraph that must be retained to symbolically generate the same filled graph (that is, the Cholesky matrix structure).

Comparisons with the popular compressed column storage scheme by Sherman [22] indicate that this implicit row scheme requires significantly less overhead storage for subscripts on many practical problems. We can, however, provide contrived matrix examples in which one scheme is substantially better than the other. Figure 8 contains two Cholesky matrix structures, the pattern of which can be easily generalized to n-by-n problems. The compact row scheme is more suitable for the first structure, whereas the compressed column scheme is more suitable for the second one. The author does not have any simple and useful characterization of matrix structures for which the row scheme works better than the column scheme or vice versa.

The advantage that the compact row scheme has over the compressed column scheme is that the number of subscripts stored is always bounded by the number

of nonzeros in the *original* matrix structure (see Theorem 3.1). Furthermore, the experimental runs also highly favor the row scheme.

However, for the row storage scheme to be truly competitive as a practical scheme, we still have to demonstrate its effective use in the numerical factorization and solution phases of the computation. The author is currently developing a complete set of routines to do the symbolic and numerical computation based on this compact row storage scheme.

REFERENCES

1. AHO, A. V., HOPCROFT, J. E., AND ULLMAN, J. D. *Data Structures and Algorithms.* Addison-Wesley, Reading, Mass., 1983.
2. DUFF, I. S. Full matrix techniques in sparse Gaussian elimination. In *Proceedings of the Dundee Conference on Numerical Analysis.* Lecture Notes in Mathematics, 912, G. A. Watson, Ed., Springer-Verlag, New York, N.Y. 1982, pp. 71–84.
3. DUFF, I. S. Parallel implementation of multifrontal methods. Technical Memorandum No. 49. Mathematics and Computer Science Division, Argonne National Laboratory, Argonne, IL, 1985.
4. DUFF, I. S., GRIMES, R., LEWIS, J., AND POOLE, B. Sparse matrix test problems. *SIGNUM Newsl.*, (June 1982), p. 22.
5. DUFF, I. S., AND REID, J. K. The multifrontal solution of indefinite sparse symmetric linear equations. *ACM Trans. Math. Softw. 9*, 3 (1983), 302–325.
6. EISENSTAT, S. C., SCHULTZ, M. H., AND SHERMAN, A. H. Applications of an element model for Gaussian elimination. In *Sparse Matrix Computations*, J. R. Bunch and D. J. Rose, Eds., Academic Press, New York, N.Y. 1976, pp. 85–96.
7. EISENSTAT, S. C., SCHULTZ, M. H., AND SHERMAN, A. H. Software for sparse Gaussian elimination with limited core storage. In *Sparse Matrix Proceedings*, I. S. Duff and G. W. Stewart, Eds., SIAM Press, Philadelphia, Pa. 1979, pp. 135–153.
8. EISENSTAT, S. C., SCHULTZ, M. H., AND SHERMAN, A. H. Algorithms and data structures for sparse symmetric Gaussian elimination. *SIAM J. Sci. Stat. Comput.*, *2*, (1981), 225–237.
9. GEORGE, J. A. Nested dissection of a regular finite element mesh. *SIAM J. Numer. Anal.*, *10*, (1973), 345–363.
10. GEORGE, J. A., AND LIU, J. W. H. A note on fill for sparse matrices. *SIAM J. Numer. Anal.*, *12*, (1975), 452–454.
11. GEORGE, J. A., AND LIU, J. W. H. An optimal algorithm for symbolic factorization of symmetric matrices. *SIAM J. Comput. 9*, (1980), 583–593.
12. GEORGE, J. A., AND LIU, J. W. H. *Computer Solution of Large Sparse Positive Definite Systems.* Prentice-Hall Inc., Englewood Cliffs, N.J., 1981.
13. JESS, J. A. G., AND KEES, H. G. M. A data structure for parallel L/U decomposition. *IEEE Trans. Comput., C-31*, (1982), 231–239.
14. LIU, J. W. H. On general row merging schemes for sparse Givens transformations. Tech. Rep. CS-83-04, Dept. of Computer Science, York University, Ontario, Canada, 1983. To appear in *SIAM J. Sci. Stat. Comput.*
15. LIU, J. W. H. Computational models and task scheduling for parallel sparse Cholesky factorization. Tech. Rep. CS-85-01, Dept. of Computer Science, York University, Ontario, Canada, 1985. To appear in *Parallel Computing.*
16. LIU, J. W. H. Modification of the minimum-degree algorithm by multiple elimination. *ACM Trans. Math Softw. 11*, 2 (1985), 141–153.
17. PARTER, S. The use of linear graphs in Gaussian elimination. *SIAM Rev. 3*, (1961), 119–130.
18. PETERS, F. J. Sparse matrices and substructures. *Mathematical Centre Tracts, 119*, Mathematisch Centrum, Amsterdam, The Netherlands, 1980.
19. ROSE, D. J. A graph-theoretic study of the numerical solution of sparse positive definite systems of linear equations. In *Graph Theory and Computing*, R. C. Read, Ed., Academic Press, New York, N.Y. 1973, 183–217.

20. ROSE, D. J., WHITTEN, G. F., SHERMAN, A. H., AND TARJAN, R. E. Algorithms and software for in-core factorization of sparse symmetric positive definite matrices. *Comput. Struct. 11*, (1980), 597–608.
21. SCHREIBER, R. A new implementation of sparse Gaussian elimination. *ACM Trans. Math Softw. 8*, (1982), 256–276.
22. SHERMAN, A. H. On the efficient solution of sparse systems of linear and nonlinear equations. Ph.D. dissertation, Dept. of Computer Science, Yale University, New Haven, Conn., 1975.
23. TARJAN, R. E. Efficiency of a good but not linear set union algorithm. *J. ACM 22*, 2, (1975), 215–225.
24. TARJAN, R. E. *Data Structures and Network Algorithms*. CBMS-NSF Regional Conference Series in Applied Math, SIAM Press, Philadelphia, Pa. 1983.

Received October 1984; revised January 1986; accepted May 1986.